Name : Praju Maharjan

Group : L5CG9

Workshop 6

1. Create a program which applies a sepia filter on an image. This is the process of recalculating each pixel's RGB values using the sepia conversion formula.

   Formula:
   NewR = 0.393R + 0.769G + 0.189B
   NewG = 0.349R + 0.686G + 0.168B
   NewB = 0.272R + 0.534G + 0.131B

   If any value exceeds 255, set it to 255.
   NOTE: 0 is minimum and 255 is maximum.

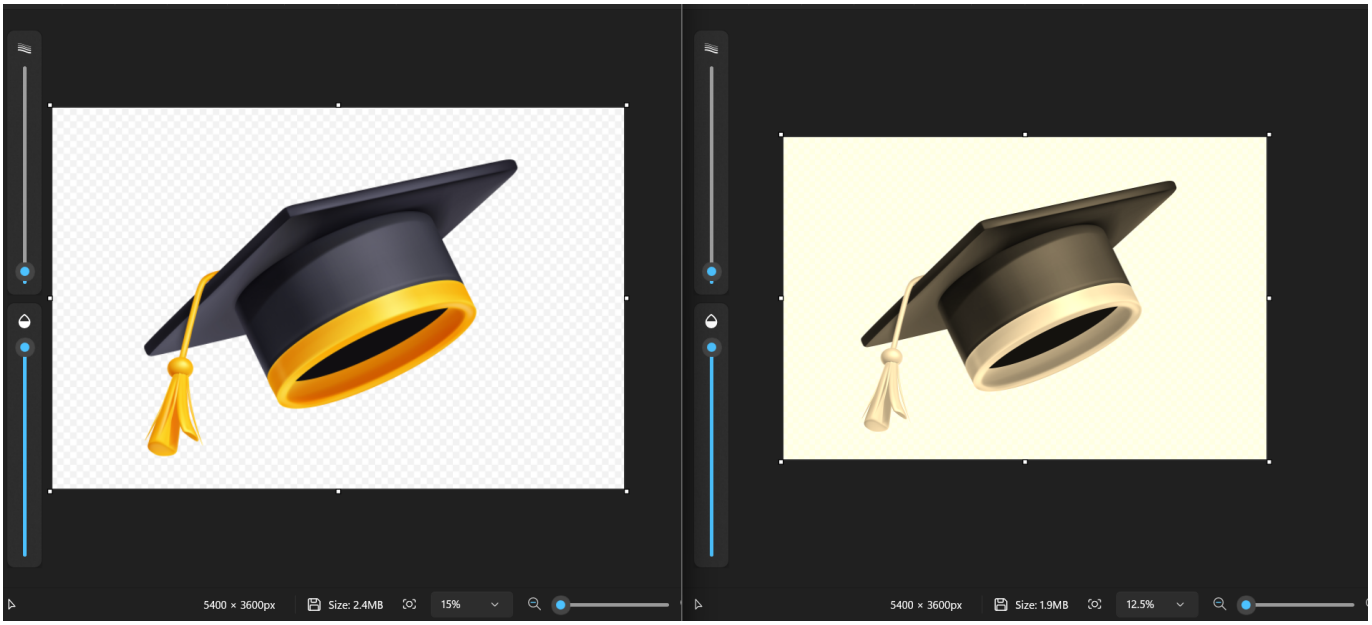
Ans :

Source Code :

```c
#include <stdio.h>
#include <stdlib.h>
#include "lodepng.h"

int main(){
        unsigned char *image;
        unsigned width,height;
        unsigned error;

        error=lodepng_decode32_file(&image,&width,&height,"image.png");
        if(error){
     printf("Error %u: %s\n", error,lodepng_error_text(error));
     return 1;
    }

        for(unsigned i=0;i<width*height*4;i+=4){
                unsigned char r= image[i];
    unsigned char g=image[i+1];
    unsigned char b=image[i+2];

    int newR=(int)(0.393*r+0.769*g+0.189*b);
    int newG=(int)(0.349*r+0.686*g+0.168*b);
    int newB=(int)(0.272*r+0.534*g+0.131*b);

        if(newR>255)newR=255;
        if(newG>255)newG=255;
        if(newB>255)newB=255;

        image[i]=newR;
        image[i+1]=newG;
        image[i+2]=newB;
    }

    error=lodepng_encode32_file("sepia.png",image,width,height);
    if(error){
        printf("Error saving image : %s\n",lodepng_error_text(error));
        free(image);
        return 1;
    }

   printf("Sepia image saved as sepia.png\n");
        free(image);
        printf("\n");
        return 0;
}
```

Output :

```
Sepia image saved as sepia.png
```



2. Create a program which rotates an image file (90/180/270 degrees) depending on the command line argument using “argv.”

Ans :

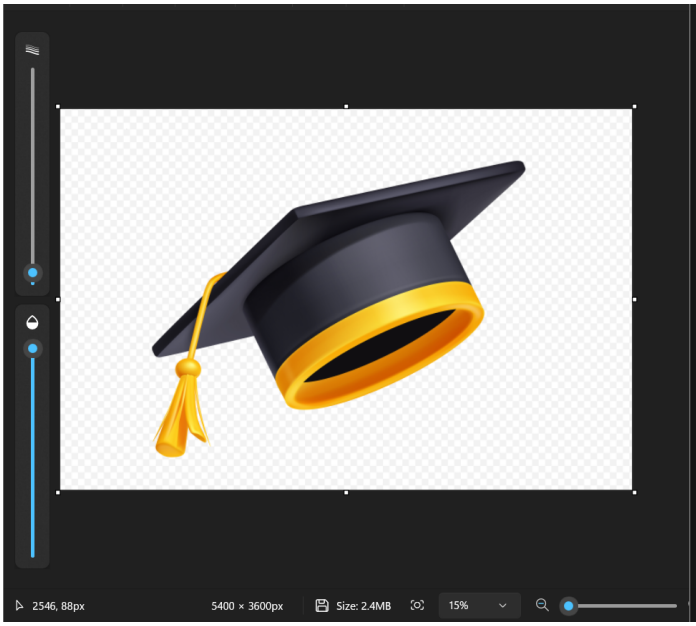
Source Code :

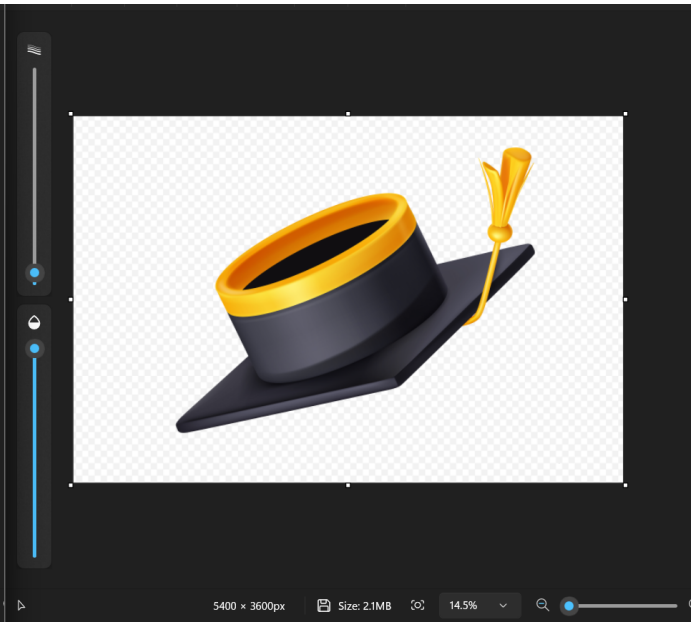
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "lodepng.h"

int main(int argc,char *argv[]){
        if(argc!=3){
                printf("Usage: %s input.png angle(90/180/270)\n", argv[0]);
    return 1;
    }

  unsigned char *image,*rotated;
  unsigned width,height,error;
  int angle=atoi(argv[2]);

  error=lodepng_decode32_file(&image,&width,&height,argv[1]);
  if(error){
        printf("Decode error %u: %s\n", error,lodepng_error_text(error));
    return 1;
  }

  unsigned newWidth=width;
  unsigned newHeight=height;

  if(angle==90||angle==270){
        newWidth=height;
        newHeight=width;
    }

  rotated=(unsigned char*) malloc(newWidth*newHeight*4);

  if(rotated==NULL){
        printf("Memory allocation failed\n");
        free(image);
        return 1;
   }

   for(unsigned y=0;y<height;y++){
      for(unsigned x=0;x<width;x++){
         unsigned oldIndex=(y*width+x)*4;
         unsigned newX,newY;

         if(angle==90){
```

```
            newX=height-1-y;
            newY=x;
         }
         else if(angle==180){
            newX=width-1-x;
            newY=height-1-y;
         }
         else if(angle==270){
            newX=y;
            newY=width-1-x;
         }
         else{
            printf("Invalid angle. Use 90, 180, or 270.\n");
            free(image);
            free(rotated);
            return 1;
            }

         unsigned newIndex=(newY*newWidth+newX)*4;

         for(int i=0;i<4;i++){
            rotated[newIndex+i]=image[oldIndex+i];
           }
       }
   }

    error=lodepng_encode32_file("rotated.png",rotated,newWidth, newHeight);
    if(error){
        printf("Encode error %u: %s\n", error, lodepng_error_text(error));
    }else{
        printf("Image rotated and saved as rotated.png\n");
    }

  free(image);
  free(rotated);
        printf("\n");
        return 0;
}
```

Output :

```
praju-maharjan@LOCKON:~/Algorithms_and_Concurrency/Workshop-6$ ./Q2 image.png 180
Image rotated and saved as rotated.png

praju-maharjan@LOCKON:~/Algorithms_and_Concurrency/Workshop-6$ ./Q2 image.png 900
Invalid angle. Use 90, 180, or 270.
```





3. Create a program which adjusts the brightness of an image file depending on the command line argument using “argv.” The user should enter a positive value to increase brightness or a negative value to decrease brightness.

   Formula:

   R = R + value
   G = G + value
   B = B + value
   If any value becomes greater than 255, set it to 255.
   If any value becomes less than 0, set it to 0.

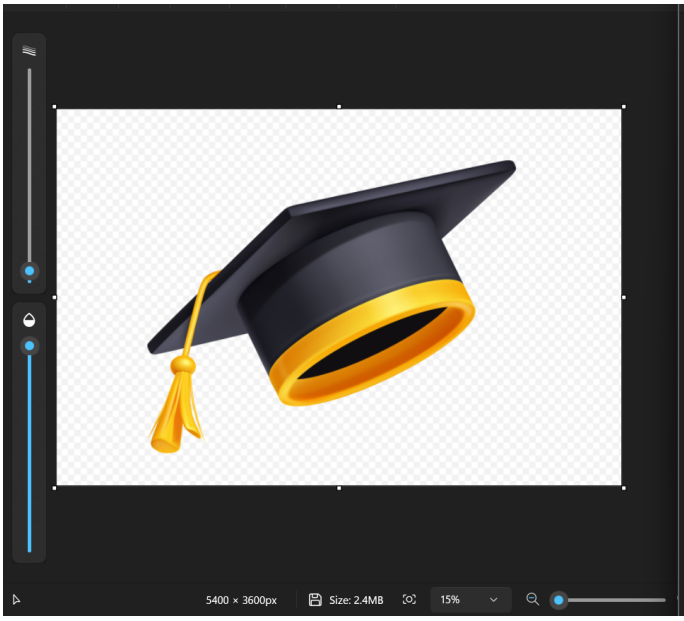
Ans :

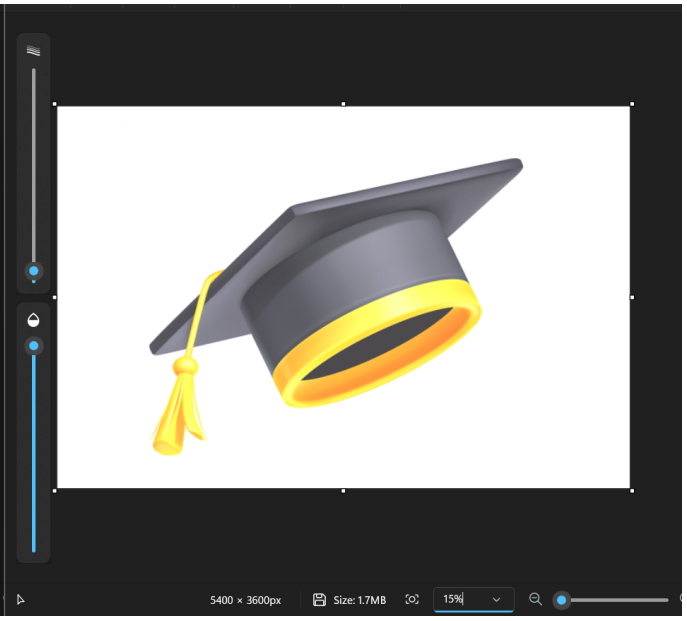
Source Code :

```c
#include <stdio.h>
#include <stdlib.h>
#include "lodepng.h"

int main(int argc, char *argv[]){
    if(argc != 3){
        printf("Usage: %s input.png brightness_value\n", argv[0]);
        return 1;
    }

    unsigned char *image;
    unsigned width, height, error;

    int value = atoi(argv[2]);

    error = lodepng_decode32_file(&image, &width, &height, argv[1]);
    if(error){
        printf("Decode error %u: %s\n", error, lodepng_error_text(error));
        return 1;
    }

    for(unsigned i = 0; i < width * height * 4; i += 4){
        int r = image[i]     + value;
        int g = image[i + 1] + value;
        int b = image[i + 2] + value;

        if(r > 255) r = 255;
        if(g > 255) g = 255;
        if(b > 255) b = 255;

        if(r < 0) r = 0;
        if(g < 0) g = 0;
        if(b < 0) b = 0;

        image[i]     = r;
        image[i + 1] = g;
        image[i + 2] = b;
    }

    error = lodepng_encode32_file("brightness.png", image, width, height);
    if(error){
        printf("Encode error %u: %s\n", error, lodepng_error_text(error));
        free(image);
        return 1;
    }

    printf("Brightness adjusted image saved as brightness.png\n");

    free(image);
    return 0;
}
```

Output :

```
praju-maharjan@LOCKON:~/Algorithms_and_Concurrency/Workshop-6$ ./Q3 image.png 60
Brightness adjusted image saved as brightness.png
```





4. Research different types of blurring techniques for images and apply at least 2 using the lodepng library.

    a. Box blur
    b. Gaussian blur

Ans :

Source Code :

```c
#include <stdio.h>
#include <stdlib.h>
#include "lodepng.h"

void box_blur(unsigned char *image, unsigned char *output, unsigned w, unsigned h){
    for(unsigned y = 1; y < h-1; y++){
        for(unsigned x = 1; x < w-1; x++){

            int sumR=0, sumG=0, sumB=0;

            for(int dy=-1; dy<=1; dy++){
                for(int dx=-1; dx<=1; dx++){
                    int idx = ((y+dy)*w + (x+dx)) * 4;
                    sumR += image[idx];
                    sumG += image[idx+1];
                    sumB += image[idx+2];
                }
            }

            int out = (y*w + x) * 4;
            output[out]   = sumR / 9;
            output[out+1] = sumG / 9;
            output[out+2] = sumB / 9;
            output[out+3] = image[out+3]; // keep alpha
        }
    }
}

void gaussian_blur(unsigned char *image, unsigned char *output, unsigned w, unsigned h){
    int kernel[3][3] = {
        {1,2,1},
        {2,4,2},
        {1,2,1}
    };

    for(unsigned y = 1; y < h-1; y++){
        for(unsigned x = 1; x < w-1; x++){

            int sumR=0, sumG=0, sumB=0;

            for(int dy=-1; dy<=1; dy++){
                for(int dx=-1; dx<=1; dx++){
                    int idx = ((y+dy)*w + (x+dx)) * 4;
                    int k = kernel[dy+1][dx+1];

                    sumR += image[idx]   * k;
                    sumG += image[idx+1] * k;
                    sumB += image[idx+2] * k;
                }
            }
```

```
            int out = (y*w + x) * 4;
            output[out]   = sumR / 16;
            output[out+1] = sumG / 16;
            output[out+2] = sumB / 16;
            output[out+3] = image[out+3];
        }
    }
}

int main(){
    unsigned char *image, *box, *gauss;
    unsigned w, h, error;

    error = lodepng_decode32_file(&image, &w, &h, "image.png");
    if(error){
        printf("Error: %s\n", lodepng_error_text(error));
        return 1;
    }

    box   = malloc(w * h * 4);
    gauss = malloc(w * h * 4);

    box_blur(image, box, w, h);
    gaussian_blur(image, gauss, w, h);

    lodepng_encode32_file("box_blur.png", box, w, h);
    lodepng_encode32_file("gaussian_blur.png", gauss, w, h);

    printf("Box blur saved as box_blur.png\n");
    printf("Gaussian blur saved as gaussian_blur.png\n");

    free(image);
    free(box);
    free(gauss);

    return 0;
}
```

Output :

```
Box blur saved as box_blur.png
Gaussian blur saved as gaussian_blur.png
```